n.16 - Przekazywanie obiektów jako parametrów funkcji

Przedmiot: programowanie aplikacji. Klasa 4 technikum, zawód technik programista. Temat utrwala przekazywanie obiektów do funkcji w języku C++ oraz łączy klasy, referencje, wskaźniki i const-correctness.

1. Problem przekazywania obiektów przez wartość

Obiekt jest instancją klasy i może zawierać wiele pól, inne obiekty oraz dane zaalokowane dynamicznie. Gdy obiekt zostanie przekazany do funkcji przez wartość, tworzona jest jego kopia. Przy dużych obiektach zwiększa to zużycie pamięci i wydłuża czas działania programu.

2. Zalecane sposoby przekazywania obiektów

Sposób	Deklaracja	Znaczenie
Przez wartość	void f(Pracownik p)	Powstaje kopia obiektu. Zmiany nie dotyczą oryginału.
Przez referencję	void f(Pracownik& p)	Funkcja pracuje na oryginalnym obiekcie i może go zmieniać.
Przez stałą referencję	void f(const Pracownik& p)	Funkcja odczytuje obiekt bez kopiowania i bez prawa do zmiany.
Przez wskaźnik	void f(Pracownik* p)	Funkcja dostaje adres obiektu. Dostęp do pól wykonuje się przez ->.

3. Parametry wejściowe i wyjściowe

Parametry wejściowe funkcji najlepiej definiować jako stałe referencje lub stałe wskaźniki, jeśli funkcja tylko czyta dane. Dzięki temu nie powstaje kopia obiektu, a funkcja nie może przypadkowo zmienić danych.

Parametry wyjściowe to parametry, przez które funkcja przekazuje wynik do otoczenia. Jeśli funkcja ma zmienić obiekt, parametr nie może być const.

Przykład - referencja

```
#include <iostream>
using namespace std;

class Pracownik {
public:
    string imie;
    string nazwisko;
};

Pracownik& pobierzDane(Pracownik& p) {
    cout << "Imie = "; cin >> p.imie;
    cout << "Nazwisko = "; cin >> p.nazwisko;
    return p;
}

void wyswietlDane(const Pracownik& p) {
    cout << "Imie: " << p.imie << endl;
    cout << "Nazwisko: " << p.nazwisko << endl;
}

int main() {
    Pracownik pracownik;
    pobierzDane(pracownik);
    wyswietlDane(pracownik);
    return 0;
}
```

Przykład - wskaźnik

```
Pracownik* pobierzDane(Pracownik* w_p) {
    cout << "Imie = "; cin >> w_p->imie;
    cout << "Nazwisko = "; cin >> w_p->nazwisko;
    return w_p;
}

void wyswietlDane(const Pracownik* w_p) {
    if (w_p != nullptr) {
        cout << "Imie: " << w_p->imie << endl;
        cout << "Nazwisko: " << w_p->nazwisko << endl;
    }
}

int main() {
    Pracownik* w_pracownik = new Pracownik();
    pobierzDane(w_pracownik);
    wyswietlDane(w_pracownik);
    delete w_pracownik;
    return 0;
}
```

4. Dobre praktyki

- Do odczytu dużych obiektów stosuj const Typ&.
- Do modyfikacji istniejącego obiektu stosuj Typ&.
- Wskaźnik stosuj, gdy potrzebny jest adres obiektu, obiekt dynamiczny lub możliwość braku obiektu.
- Dla obiektu utworzonego przez new pamiętaj o delete.
- Przy wskaźnikach stosuj operator ->, a przy obiektach i referencjach operator kropki .

5. Skróty i pojęcia

const - modyfikator oznaczający brak prawa do zmiany wartości. RAM - pamięć operacyjna komputera. nullptr - wartość wskaźnika oznaczająca, że nie wskazuje on żadnego obiektu. OOP (Object-Oriented Programming) - programowanie obiektowe.